SWIGGY SALES ANALYSIS

IMPORTING LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

IMPORTING DATA

```
df = pd.read_excel("/content/swiggy_data.xlsx")
```

UNDERSTANDING THE DATA

```
df.head()
```

	State	City	Order Date	Restaurant Name	Location	Category	Dis Nam
0	Karnataka	Bengaluru	2025-06-29	Anand Sweets & Savouries	Rajarajeshwari Nagar	Snack	Butt Murukk 200g
1	Karnataka	Bengaluru	2025-04-03	Srinidhi Sagar Deluxe	Kengeri	Recommended	Bada Mi
2	Karnataka	Bengaluru	2025-01-15	Srinidhi Sagar Deluxe	Kengeri	Recommended	Cho Cho Ba
3	Karnataka	Bengaluru	2025-04-17	Srinidhi Sagar Deluxe	Kengeri	Recommended	Kesa Ba
4	Karnataka	Bengaluru	2025-03-13	Srinidhi Sagar Deluxe	Kengeri	Recommended	M Raith

```
df.tail()
```

	State	City	Order Date	Restaurant Name	Location	Category	Dish Name	Price (INR)
197425	Sikkim	Gangtok	2025-01-25	Mama's Kitchen	Gangtok	Momos	Soya cheese chilli momo ...	112.0
197426	Sikkim	Gangtok	2025-07-02	Mama's Kitchen	Gangtok	Momos	Kurkure momo fried ...	140.0
197427	Sikkim	Gangtok	2025-03-25	Mama's Kitchen	Gangtok	Momos	Chilli cheese momo	126.0
197428	Sikkim	Gangtok	2025-03-26	Mama's Kitchen	Gangtok	Momos	Veg Momos (8 Pc)	85.0
197429	Sikkim	Gangtok	2025-03-27	Mama's Kitchen	Gangtok	Momos	Soya Momo	100.0

```
print("No. of Rows:" , df.shape[0])
```

```
No. of Rows: 197430
```

```
print("No. of Fields:" , df.shape[1])
```

```
No. of Fields: 10
```

```
df.info
```

pandas.core.frame.DataFrame.info
```
def info(verbose: bool | None=None, buf: WriteBuffer[str] | None=None,
max_cols: int | None=None, memory_usage: bool | str | None=None,
show_counts: bool | None=None) -> None
```

```
Print a concise summary of a DataFrame.

This method prints information about a DataFrame including
the index dtype and columns, non-null values and memory usage.

Parameters
```

DATA TYPES

```
df.dtypes
```

	0
State	object
City	object
Order Date	datetime64[ns]
Restaurant Name	object
Location	object
Category	object
Dish Name	object
Price (INR)	float64
Rating	float64
Rating Count	int64

dtype: object

```
df.describe()
```

	Order Date	Price (INR)	Rating	Rating Count
count	197430	197430.000000	197430.000000	197430.000000
mean	2025-05-01 19:41:20.996808960	268.512920	4.341582	28.321805
min	2025-01-01 00:00:00	0.950000	1.500000	0.000000
25%	2025-03-01 00:00:00	139.000000	4.300000	0.000000
50%	2025-05-02 00:00:00	229.000000	4.400000	2.000000
75%	2025-07-01 00:00:00	329.000000	4.500000	15.000000
max	2025-08-31 00:00:00	8000.000000	5.000000	999.000000
std	NaN	219.338363	0.422585	87.542593

CLEAN & PREPARE THE DATA

Remove duplicates

```
df = df.drop_duplicates()
```

Label each dish as Veg or Non-Veg

```
non_veg_keywords = [
    "chicken", "egg", "fish" , "mutton",
    "prawn", "biryani","kabab","kebab",
    "non-veg","non veg"
]
df["Food Category"] = np.where(
    df["Dish Name"].str.lower().str.contains('|'.join(non_veg_keywords),na=F
    "Non-Veg",
    "Veg"
)
```

KPI's

Total Sales

```
total_sales = df["Price (INR)"].sum()
print("Total Sales (INR):" ,round(total_sales,2))
```

```
Total Sales (INR): 53012505.77
```

Average Rating

```
average_rating = df["Rating"].mean()
print("Average Rating :" ,round(average_rating,1))
```

```
Average Rating : 4.3
```

Average Order Value

```
average_order_value = df["Price (INR)"].mean()
print("Average Order Value (INR) :" ,round(average_order_value,2))
```

```
Average Order Value (INR) : 268.51
```

Ratings Count

```
ratings_count = df["Rating Count"].sum()
print("Ratings Count :" ,round(ratings_count,2))
```

```
Ratings Count : 5591574
```

Total Orders

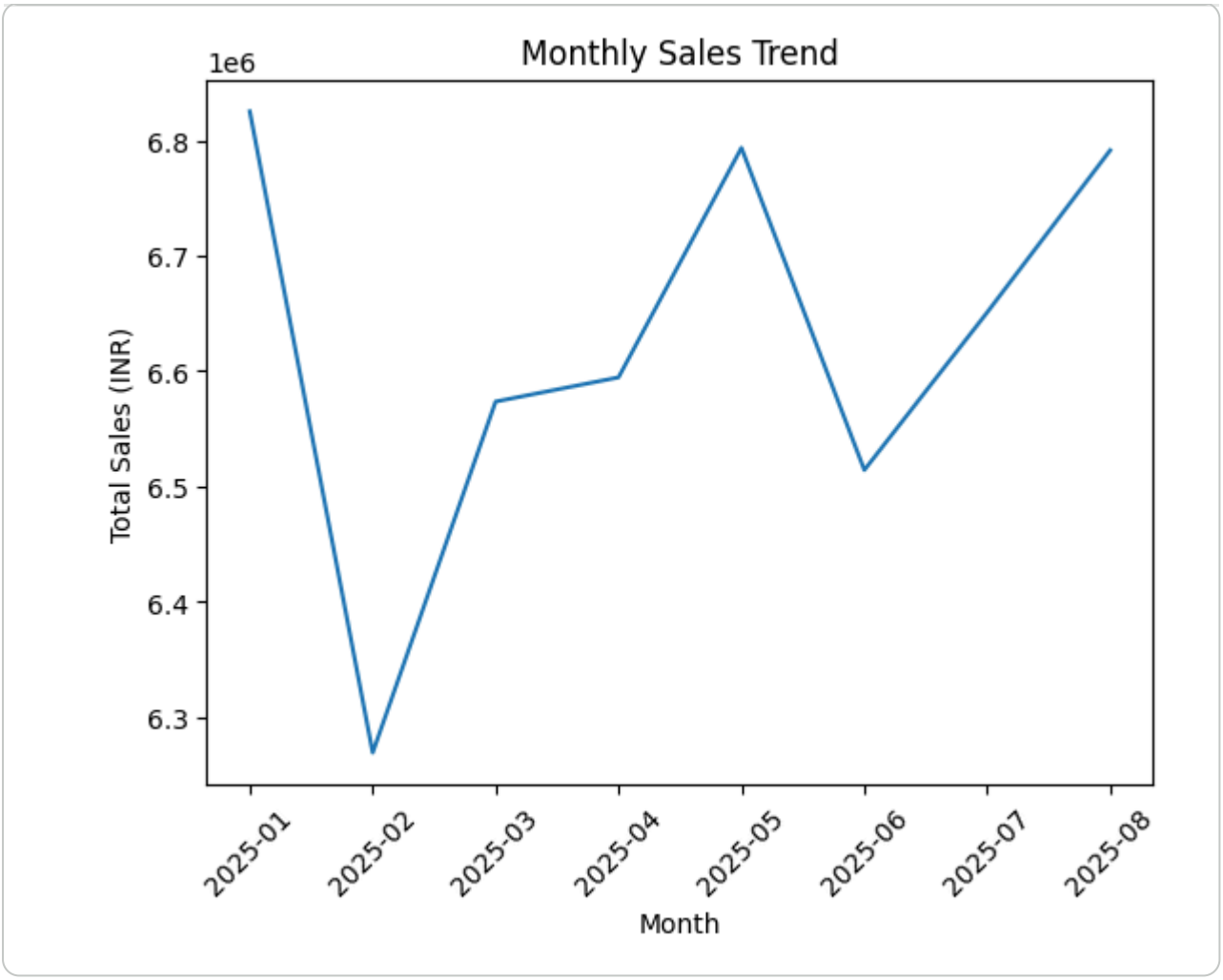
```
total_orders = (len(df))
print("Total Orders :" ,round(total_orders,2))
```

```
Total Orders : 197430
```

Charts Design

Monthly Sales Trend

```
df["Order Date"] = pd.to_datetime(df["Order Date"])

df["YearMonth"] = df["Order Date"].dt.to_period("M").astype(str)

monthly_revenue = df.groupby("YearMonth")["Price (INR)"].sum().reset_index()

plt.figure()
plt.plot(monthly_revenue["YearMonth"],monthly_revenue["Price (INR)"])
plt.xticks(rotation = 45)
plt.xlabel("Month")
plt.ylabel("Total Sales (INR)" )
plt.title("Monthly Sales Trend")
plt.tight_layout
plt.show()
```



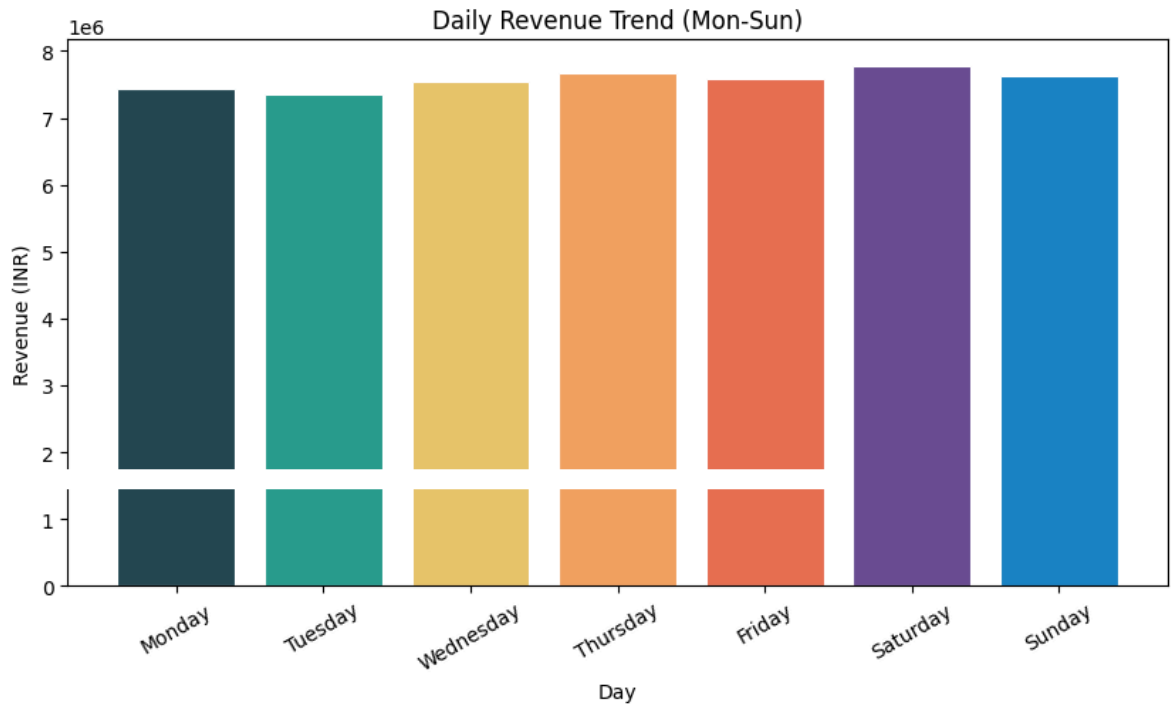
Interpretation:

1. The monthly sales trend indicates fluctuations in customer ordering patterns across different months.
2. January 2025 was the peak month at ₹68.2L. February dipped to ₹62.7L simply because it has fewer days not a real drop. Revenue then stayed stable between ₹65-68L for all remaining months.

Daily Sales Trend

```
df["DayName"] = pd.to_datetime(df["Order Date"]).dt.day_name()

daily_revenue = (
    df.groupby("DayName")["Price (INR)"]
    .sum()
    .reindex(["Monday","Tuesday","Wednesday","Thursday","Friday","Saturday",
)
colors = ["#264653","#2A9D8F","#E9C46A","#F4A261","#E76F51","#6A4C93","#1982

plt.figure(figsize=(10,5))
bars = plt.bar(daily_revenue.index, daily_revenue.values, color=colors,edgec
plt.title("Daily Revenue Trend (Mon-Sun)")
plt.xlabel("Day")
```

```
plt.ylabel("Revenue (INR)")
plt.xticks(rotation = 30)

plt.show()
```



Interpretation:

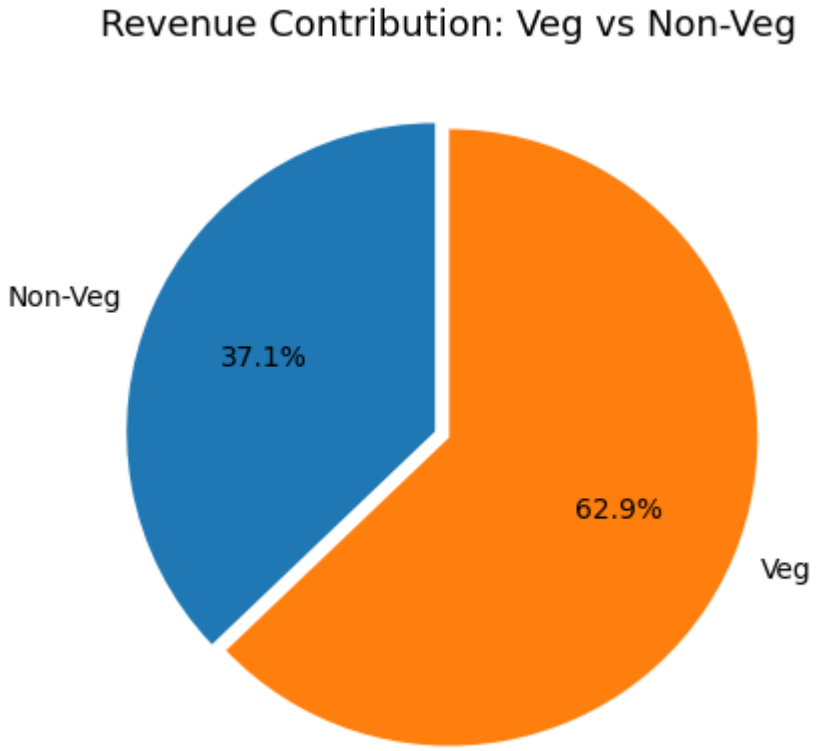
1. Revenue varies across weekdays and weekends. Revenue is strong every day of the week. Saturday is the peak and Tuesday the lowest, but the difference is only ₹4.2L — showing Swiggy's demand is daily, not just weekend-driven.
2. An increase in weekdays can be because of busy life style and working culture making it difficult to prepare food at homes in citiesand during weekends due to leisure time and social gatherings.

Total Sales by Food Type (Veg vs Non-Veg)

```
food_revenue = (
    df.groupby("Food Category")["Price (INR)"]
    .sum()
    .reset_index()
)
```

```
import matplotlib.pyplot as plt

sizes = food_revenue.groupby("Food Category")["Price (INR)"].sum()

fig, ax = plt.subplots(figsize=(7, 5))

ax.pie(
    sizes,
```

```
    labels=sizes.index,
    autopct="%1.1f%%",
    startangle=90,
    explode=[0.05 if i == 0 else 0 for i in range(len(sizes))]
)

ax.set_title("Revenue Contribution: Veg vs Non-Veg", fontsize=13)
plt.show()
```



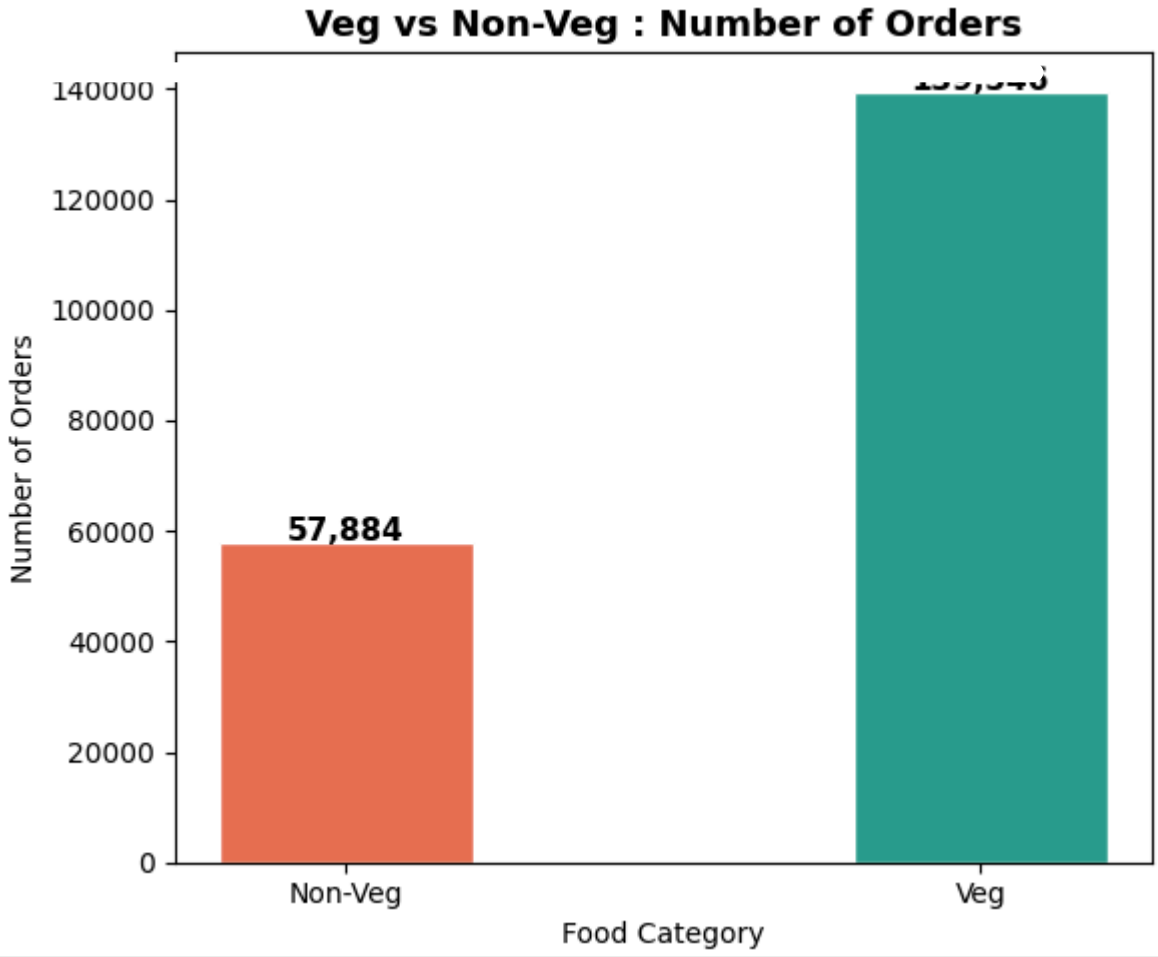
Interpretation:

1. The chart compares revenue contribution between Veg and Non-Veg categories.
2. Higher contribution from a category reflects stronger customer demand and spending behavior.
3. It is observed that people preferred more of Veg food.This also aligns with restaurant popularity trends observed in later analyses.
4. Veg contributes 62.9% of revenue from 70.7% of orders. Non-Veg contributes 37.1% from only 29.3% of orders, meaning Non-Veg customers spend more per order on average as compared to Vegetrian food.

Veg vs Non-Veg : Number of Orders

```
food_orders = df.groupby("Food Category")["Price (INR)"].count()
```

```
plt.figure(figsize=(6, 5))
bars = plt.bar(food_orders.index, food_orders.values,
               color=["#E76F51","#2A9D8F"], edgecolor="white", width=0.4)
plt.title("Veg vs Non-Veg : Number of Orders", fontsize=13, fontweight="bold
plt.xlabel("Food Category", fontsize=10)
plt.ylabel("Number of Orders", fontsize=10)

# Add labels on top of each bar
for bar in bars:
    plt.text(
        bar.get_x() + bar.get_width() / 2,   # center of the bar (left + hal
        bar.get_height() + 200,               # just above the bar
        f"{int(bar.get_height()):,}",         # the number with comma format
        ha="center", fontsize=11, fontweight="bold"
    )

plt.tight_layout()
plt.show()
```

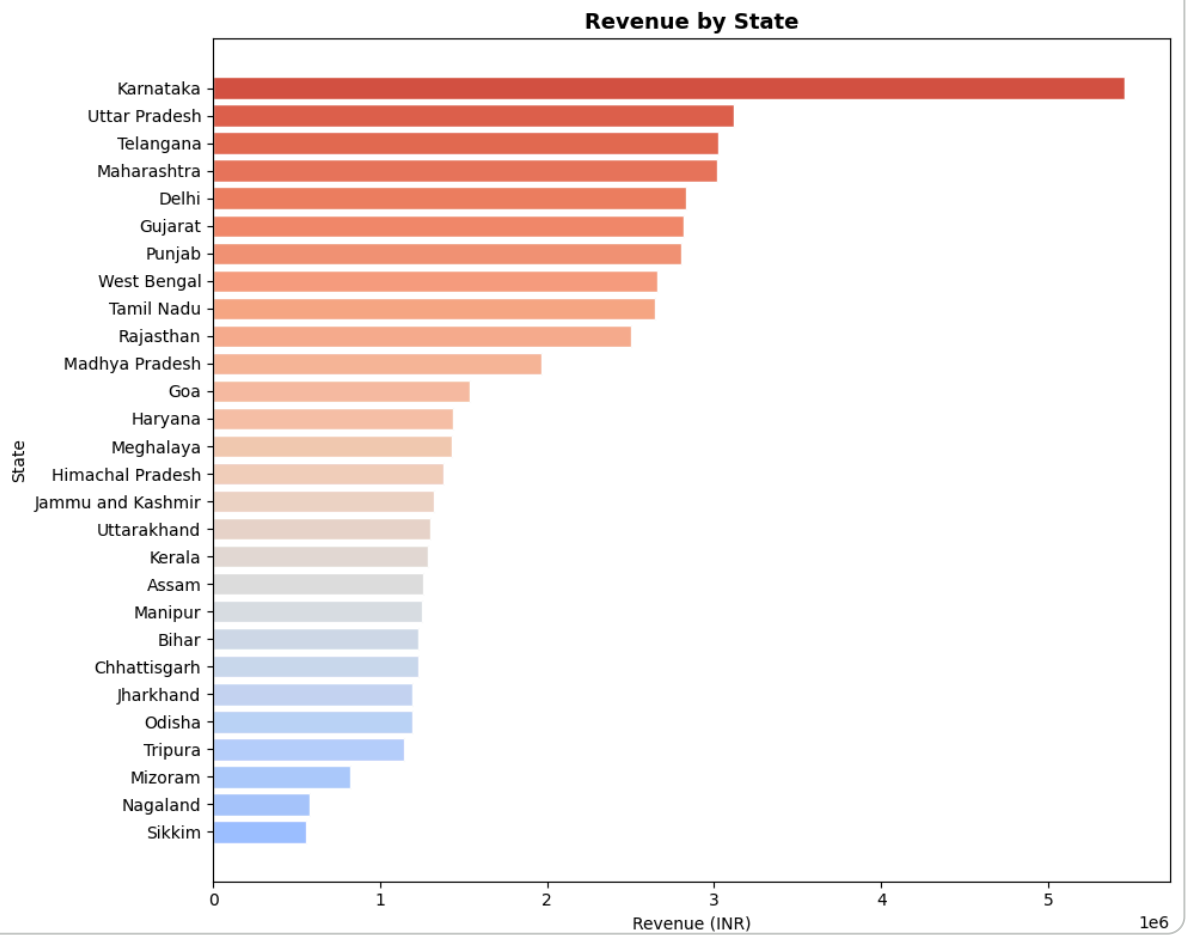


Interpretation:

1. This chart focuses on order volume rather than revenue.
2. A category may have more orders but lower revenue if the average order value is smaller.
3. It is observed that the number of Veg order is 139,546 is higher as compared to non-veg 57,884.

Total Sales by State

```
state_revenue = (
    df.groupby("State")["Price (INR)"]
    .sum()
    .sort_values()
)

colors = cm.coolwarm(np.linspace(0.3, 0.9, len(state_revenue)))

plt.figure(figsize=(10, 8))
plt.barh(state_revenue.index, state_revenue.values, color=colors, edgecolor=

plt.title("Revenue by State", fontsize=13, fontweight="bold")
plt.xlabel("Revenue (INR)", fontsize=10)
plt.ylabel("State", fontsize=10)
plt.tight_layout()
plt.show()
```



Interpretation:

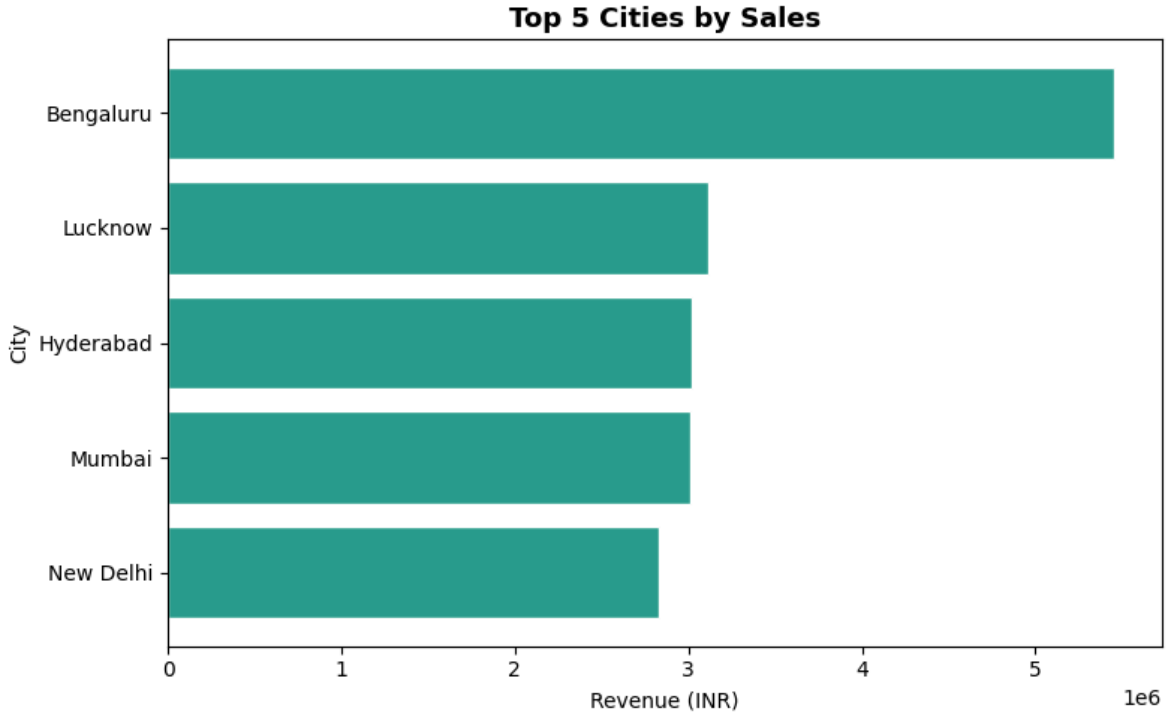
1. Karnataka records the highest revenue among all states, indicating strong customer demand.
2. In urban and IT-driven regions. Metropolitan and working-class populations contribute heavily to online food delivery growth, which is further supported in

the city-wise sales analysis.

3. Sikkim records the lowest revenue which is expected given its small population and limited urban centres compared to larger states

Top 5 Cities by Sales

```
top_5_cities = (
    df.groupby("City")["Price (INR)"]
    .sum()
    .sort_values(ascending=True)
    .tail(5)
)

plt.figure(figsize=(8, 5))
plt.barh(top_5_cities.index, top_5_cities.values, color="#2A9D8F", edgecolor

plt.title("Top 5 Cities by Sales", fontsize=13, fontweight="bold")
plt.xlabel("Revenue (INR)", fontsize=10)
plt.ylabel("City", fontsize=10)
plt.tight_layout()
plt.show()
```



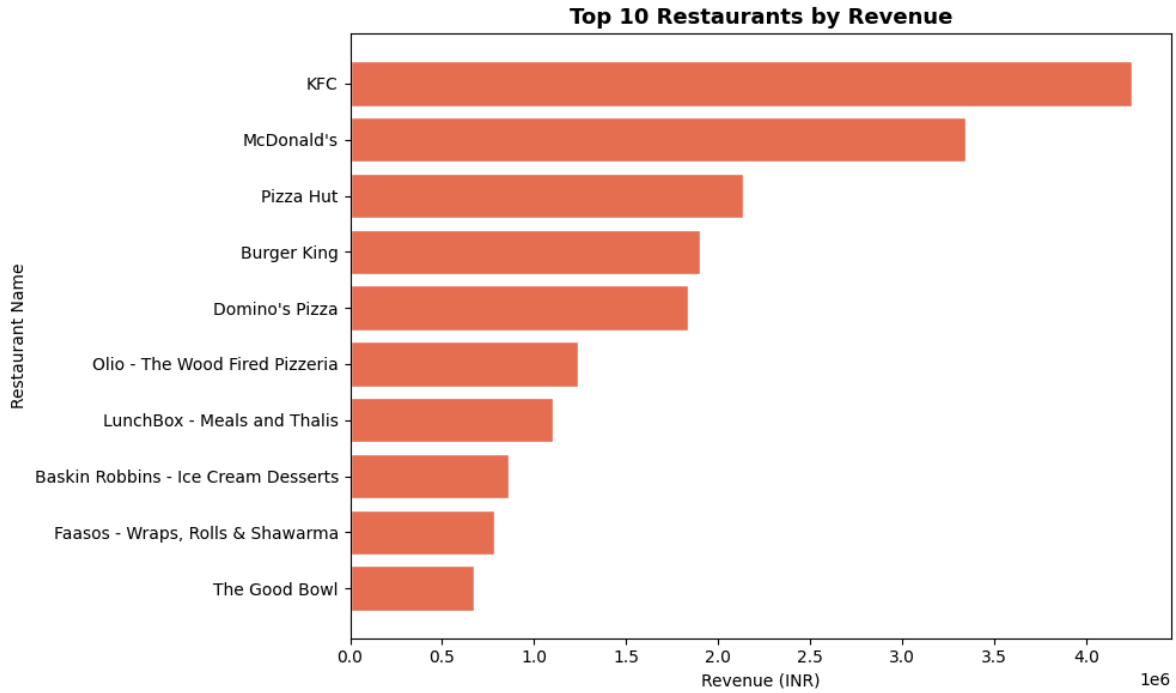
Interpretation:

1. Bengaluru records the highest sales among cities, supporting the earlier finding that Karnataka generated the highest state-wise revenue. As a major IT and working-class hub,customers in such cities prefer convenient online food delivery services due to busy lifestyles.

2. It is observed that in All the 5 cities are majority of the population is working leading to high demand in such cities.
3. Notably, Lucknow ranks 2 ahead of major metros like Mumbai and Hyderabad, showing that Tier-2 cities with young working populations are becoming strong food delivery markets

Top 10 Restaurants by Revenue

```
top_restaurants = (
    df.groupby("Restaurant Name")["Price (INR)"]
    .sum()
    .sort_values(ascending=True)
    .tail(10)
)

plt.figure(figsize=(10, 6))
plt.barh(top_restaurants.index, top_restaurants.values, color="#E76F51", edg
plt.title("Top 10 Restaurants by Revenue", fontsize=13, fontweight="bold")
plt.xlabel("Revenue (INR)", fontsize=10)
plt.ylabel("Restaurant Name", fontsize=10)
plt.tight_layout()
plt.show()
```



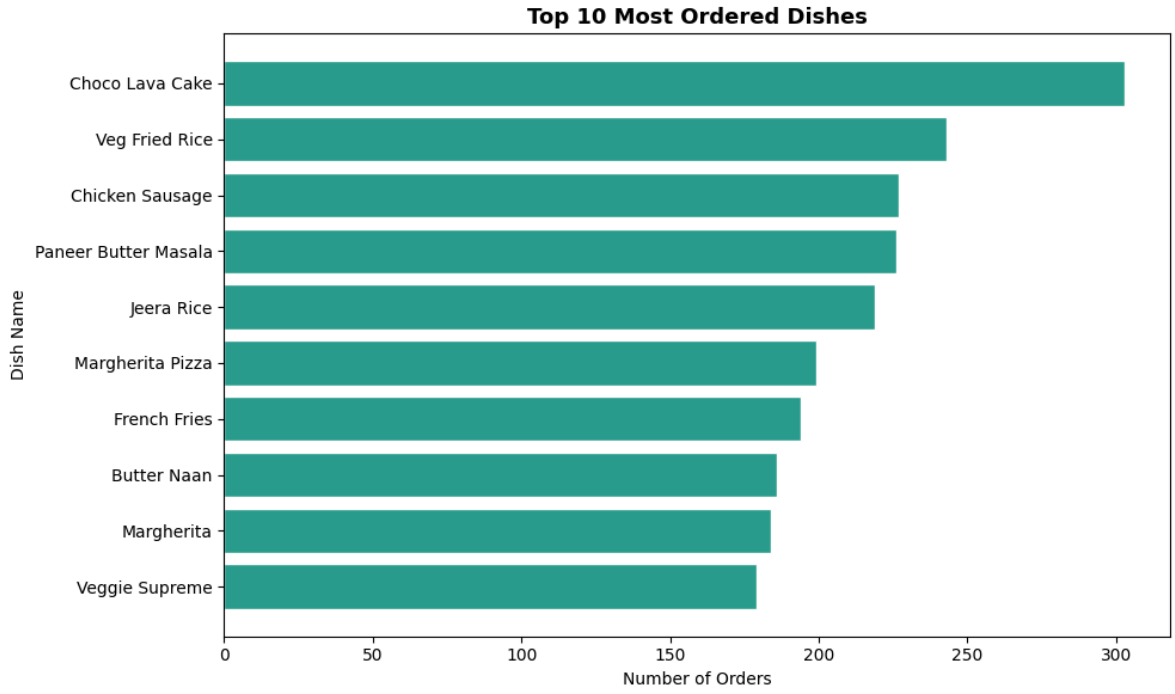
Interpretation:

1. Restaurants such as KFC, McDonald's, and Pizza Hut generate high revenue due to strong demand for fast-food options in metropolitan cities.
2. Urban customers, especially working professionals, prefer quick and convenient meal choices, supporting the city-wise sales trends observed earlier.

3. Its observed that the top 5 restaurants account for major part of total platform revenue.

Top 10 Most Ordered Dishes

```
top_dishes = (
    df.groupby("Dish Name")["Price (INR)"]
    .count()
    .sort_values(ascending=True)
    .tail(10)
)

plt.figure(figsize=(10, 6))
plt.barh(top_dishes.index, top_dishes.values, color="#2A9D8F", edgecolor="wh
plt.title("Top 10 Most Ordered Dishes", fontsize=13, fontweight="bold")
plt.xlabel("Number of Orders", fontsize=10)
plt.ylabel("Dish Name", fontsize=10)
plt.tight_layout()
plt.show()
```



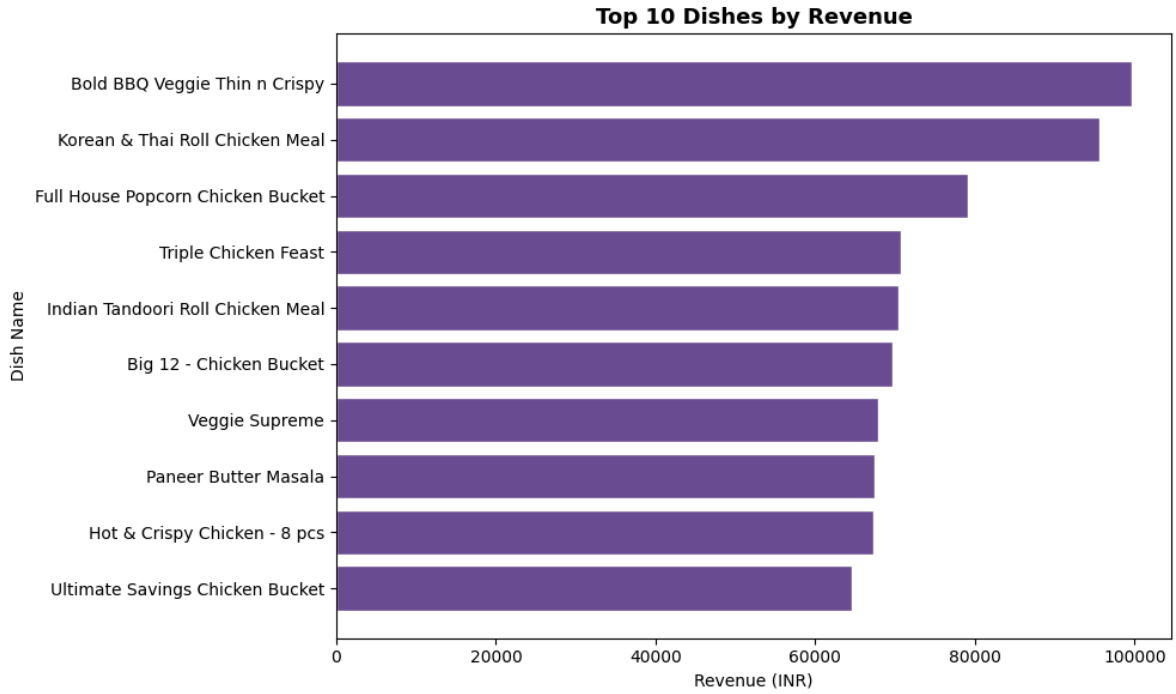
Interpretation:

1. The most ordered dishes highlight customer food preferences and repeat ordering behavior.
2. Frequently ordered items are generally affordable, convenient, and widely popular among customers, which also supports the strong performance of fast-food restaurant chains.
3. Most Ordered is Choco Lava Cake, Veg Fried Rice, Chhicken Sausage.

4. Notably, Choco Lava Cake is the most ordered item, suggesting customers regularly add desserts as an upsell to their main orders.

Top 10 Dishes by Revenue

```
top_dishes_revenue = (
    df.groupby("Dish Name")["Price (INR)"]
    .sum()
    .sort_values(ascending=True)
    .tail(10)
)

plt.figure(figsize=(10, 6))
plt.barh(top_dishes_revenue.index, top_dishes_revenue.values,
         color="#6A4C93", edgecolor="white")
plt.title("Top 10 Dishes by Revenue", fontsize=13, fontweight="bold")
plt.xlabel("Revenue (INR)", fontsize=10)
plt.ylabel("Dish Name", fontsize=10)
plt.tight_layout()
plt.show()
```



Interpretation:

1. This chart highlights dishes generating the highest revenue. Some dishes may contribute more revenue despite lower order frequency due to premium pricing or larger order values.
2. Comparing this with the most ordered dishes helps analyze customer spending behavior.

3. Choco Lava Cake, the most ordered dish, does not appear here at all. The top revenue dishes are premium combo meals priced.
4. This proves that order volume and revenue are two separate metrics — always analyse both.

Quarterly Performance Summary

```
df["Order Date"] = pd.to_datetime(df["Order Date"])

df["Quarter"] = df["Order Date"].dt.to_period("Q").astype(str)

quarterly_summary = (
    df.groupby("Quarter", as_index=False)
    .agg(
        Total_Sales=("Price (INR)", "sum"),
        Avg_Rating=("Rating", "mean"),
        Total_Orders=("Order Date", "count")
    )
    .sort_values("Quarter")
)

quarterly_summary
```

	Quarter	Total_Sales	Avg_Rating	Total_Orders
0	2025Q1	19667821.77	4.342643	73096
1	2025Q2	19902256.59	4.340011	74163
2	2025Q3	13442427.41	4.342359	50171

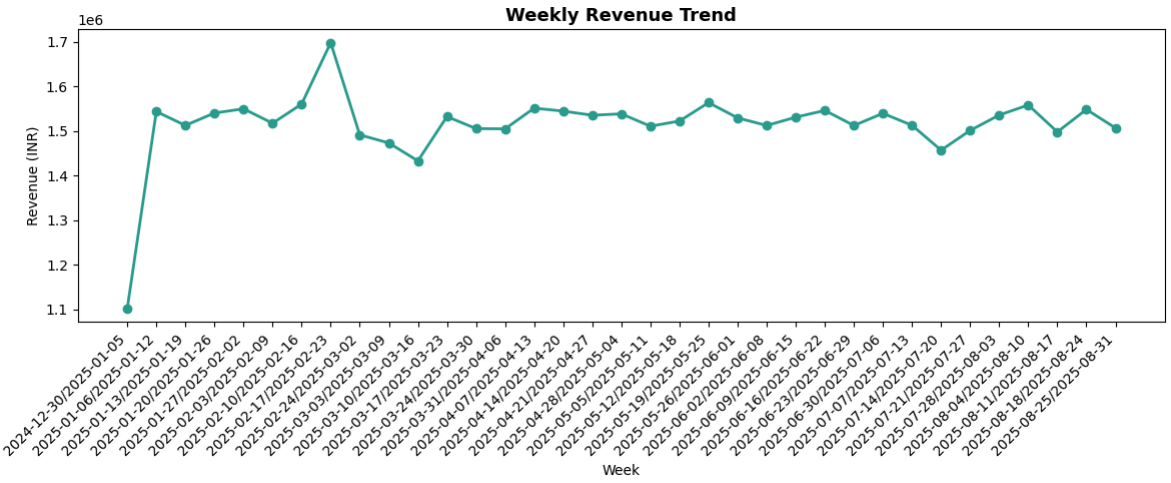
Interpretation:

1. Q1 oders were 73096 was was higher as compared to Q3 orders 50171.
2. We can also see that sales in Q1 is 19667821.77 as compared to 13442427.41 in Q3.
3. Q3 appears lower at ₹1.34Cr but the data only covers July-August (2 of 3 months). Projecting to a full quarter gives ₹2.01Cr, virtually identical to Q1 and Q2.
4. Average rating stays at 4.34 throughout, showing no dip in service quality across the year.

Weekly Trend Analysis

```
df["Order Date"] = pd.to_datetime(df["Order Date"])
df["Week"] = df["Order Date"].dt.to_period("W").astype(str)
```

```
weekly_revenue = (
    df.groupby("Week")["Price (INR)"]
    .sum()
    .reset_index()
)

plt.figure(figsize=(12, 5))
plt.plot(weekly_revenue["Week"], weekly_revenue["Price (INR)"],
         color="#2A9D8F", linewidth=2, marker="o")

plt.title("Weekly Revenue Trend", fontsize=13, fontweight="bold")
plt.xlabel("Week")
plt.ylabel("Revenue (INR)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



Interpretation:

1. Weekly revenue trends help identify short-term fluctuations in customer ordering behavior.
2. Revenue stays remarkably consistent between ₹13L–₹18L per week with no major crashes, confirming Swiggy has a stable, repeat customer base rather than a trend-dependent one
3. This analysis complements the monthly and daily sales trends observed earlier.

Key Findings & Insights

1. Karnataka Generated the Highest Revenue — by 75%

Karnataka earned more than Uttar Pradesh, the second-highest state. This is driven entirely by Bengaluru, India's tech hub, where working professionals order food daily as a necessity, not a luxury.

2. Bengaluru is the Backbone of the Entire Business

Bengaluru alone generated more than ₹50.0L, matching Karnataka's full state total. The top 5 cities (Bengaluru, Lucknow, Hyderabad, Mumbai, New Delhi) are all large metros. Food delivery demand is concentrated wherever working populations are densest.

3. 25% of Revenue Comes From Just 5 Fast-Food Chains

KFC, McDonald's, Pizza Hut, Burger King, and Domino's together account for about 25.0% of total revenue. All 5 are fast-food brands. Urban working professionals choose speed, consistency, and brand familiarity over variety.

4. Non-Veg Customers Spend More Per Order

Veg = 70.7% of orders, 62.9% of revenue. Non-Veg = 29.3% of orders, 37.1% of revenue. Non-Veg average order value is more as compared to Veg. This shows Non-Veg is the premium segment, fewer customers, but each one is worth significantly more per transaction.

5. Most Popular Dish ≠ Most Profitable Dish

Choco Lava Cake tops the orders chart. It does not appear in the top 10 revenue list.